



Test Environment & Infrastructure Specification

Dockerized Staging, Hardware Scanners, Seed Data & CI/CD Pipelines

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
DevOps & QA	QA Engineers, DevOps, Infrastructure	v1.4.0 Production	✔ Verified & Approved

Test Environment Specification

This document details the test environment setup for the StockWhisk SaaS project, including the tech stack, environment types, required tools, and network access points.

Tech Stack

The StockWhisk platform is built using a modern, robust tech stack designed for performance, scalability, and maintainability.

- **Backend:** Python 3.11+, Django 4.2+, Django REST Framework 3.14+
- **Frontend:** Node.js 18+, Next.js 14.2, React 18
- **Database:** PostgreSQL 15+ (Production/Staging), SQLite (Local Development)
- **Cache/Queue:** Redis 7+ (used as Celery broker and caching layer)
- **Deployment:** Docker Compose managing the following services:
 - `backend` (Django application)
 - `frontend` (Next.js application)
 - `worker` (Celery background task worker)
 - `beat` (Celery Beat for scheduled tasks)
 - `caddy` (Reverse proxy for SSL and routing)
 - `postgres` (Database)
 - `redis` (Cache/Broker)
- **CI/CD:** GitHub Actions (for automated testing and deployment)

Environment Types

The system supports multiple environment types to facilitate development, testing, and production deployment.

1. Local Development

- **Purpose:** Rapid iteration and local debugging for developers.
- **Setup:** Uses SQLite for the database, Django development server (`runserver`), and Next.js development server.

- **Characteristics:** Hot-reloading enabled, simplified setup, minimal dependencies.

2. Docker Development

- **Purpose:** Local testing in an environment that closely mirrors production.
- **Setup:** Uses `docker-compose up` to orchestrate all services including PostgreSQL and Redis.
- **Characteristics:** Containerized isolation, consistent environment across developer machines.

3. Staging

- **Purpose:** Pre-production testing, QA, and client previews.
- **Setup:** Production-like Docker deployment on a staging VPS.
- **Characteristics:** Uses production database configuration, representative data, accessible via a staging subdomain.

4. Production

- **Purpose:** Live environment for end-users.
- **Setup:** Docker Compose on an Ubuntu VPS, utilizing Caddy for automatic SSL/HTTPS provisioning.
- **Characteristics:** **HIGH** availability, secure, optimized for performance.

Required Tools

To effectively test and develop on the StockWhisk platform, the following tools are required:

Software Dependencies

- Python 3.11+
- `pip` (Python package installer)
- `virtualenv` (or similar virtual environment manager)
- Node.js 18+
- `npm` (Node package manager)
- Docker & Docker Compose
- Git

Hardware/Peripheral Testing Tools

- **Barcode Scanner:** USB or Bluetooth barcode scanner for testing POS input and inventory management.
- **Thermal Printer:** 80mm ESC/POS compatible thermal printer for testing receipt generation and printing layouts.

Environment Setup Steps

Backend Setup (Local Development)

1. Create and activate a virtual environment: `python -m venv venv && source venv/bin/activate`
2. Install dependencies: `pip install -r requirements.txt`
3. Apply database migrations: `python manage.py migrate`
4. Create a superuser account: `python manage.py createsuperuser`

5. Run the development server: `python manage.py runserver`

Frontend Setup (Local Development)

1. Navigate to the frontend directory.
2. Install dependencies: `npm install`
3. Run the development server: `npm run dev`

Docker Setup (Development/Staging/Production)

1. Ensure Docker and Docker Compose are installed and running.
2. Build and start the containers in detached mode: `docker compose up -d --build`

Network & Access Points

When running the environment locally (either via dev servers or Docker), the services map to the following endpoints:

- **Backend API Base URL:** `http://localhost:8000/api/`
- **Frontend Application:** `http://localhost:3000`
- **Django Admin Panel:** `http://localhost:8000/admin/`

Note: In staging or production environments, replace `localhost` and the respective ports with the configured domain names.